Lecture Notes for Chapter 4: Divide-and-Conquer

Chapter 4 overview

Recall the divide-and-conquer paradigm, which we used for merge sort:

Divide the problem into a number of subproblems that are smaller instances of the same problem.

Conquer the subproblems by solving them recursively.
Base case: If the subproblems are small enough, just solve them by brute force.

Combine the subproblem solutions to give a solution to the original problem.

We look at two more algorithms based on divide-and-conquer.

Analyzing divide-and-conquer algorithms

Use a recurrence to characterize the running time of a divide-and-conquer algorithm. Solving the recurrence gives us the asymptotic running time.

A ***recurrence*** is a function is defined in terms of

- one or more base cases, and
- itself, with smaller arguments.

Examples

- $T(n) = \begin{cases} 1 & \text{if } n = 1\,, \\ T(n-1) + 1 & \text{if } n > 1\,. \end{cases}$
  Solution: $T(n) = n$.
- $T(n) = \begin{cases} 1 & \text{if } n = 1\,, \\ 2T(n/2) + n & \text{if } n \geq 1\,. \end{cases}$
  Solution: $T(n) = n \lg n + n$.
- $T(n) = \begin{cases} 0 & \text{if } n = 2\,, \\ T(\sqrt{n}) + 1 & \text{if } n > 2\,. \end{cases}$
  Solution: $T(n) = \lg \lg n$.

- $T(n) = \begin{cases} 1 & \text{if } n = 1 , \\ T(n/3) + T(2n/3) + n & \text{if } n > 1 . \end{cases}$

  Solution: $T(n) = \Theta(n \lg n)$.

[The notes for this chapter are fairly brief because we teach recurrences in much greater detail in a separate discrete math course.]

Many technical issues:

- Floors and ceilings

  [Floors and ceilings can easily be removed and don't affect the solution to the recurrence. They are better left to a discrete math course.]
- Exact vs. asymptotic functions
- Boundary conditions

In algorithm analysis, we usually express both the recurrence and its solution using asymptotic notation.

- Example: $T(n) = 2T(n/2) + \Theta(n)$, with solution $T(n) = \Theta(n \lg n)$.
- The boundary conditions are usually expressed as "$T(n) = O(1)$ for sufficiently small n."
- When we desire an exact, rather than an asymptotic, solution, we need to deal with boundary conditions.
- In practice, we just use asymptotics most of the time, and we ignore boundary conditions.

[In my course, there are only two acceptable ways of solving recurrences: the substitution method and the master method. Unless the recursion tree is carefully accounted for, I do not accept it as a proof of a solution, though I certainly accept a recursion tree as a way to generate a guess for substitution method. You may choose to allow recursion trees as proofs in your course, in which case some of the substitution proofs in the solutions for this chapter become recursion trees.

I also never use the iteration method, which had appeared in the first edition of Introduction to Algorithms. I find that it is too easy to make an error in parenthesization, and that recursion trees give a better intuitive idea than iterating the recurrence of how the recurrence progresses.]

Maximum-subarray problem

Input: An array $A[1 \,..\, n]$ of numbers. *[Assume that some of the numbers are negative, because this problem is trivial when all numbers are nonnegative.]*

Output: Indices i and j such that $A[i \,..\, j]$ has the greatest sum of any nonempty, contiguous subarray of A, along with the sum of the values in $A[i \,..\, j]$.

Scenario

- You have the prices that a stock traded at over a period of n consecutive days.
- When should you have bought the stock? When should you have sold the stock?
- Even though it's in retrospect, you can yell at your stockbroker for not recommending these buy and sell dates.

To convert to a maximum-subarray problem, let

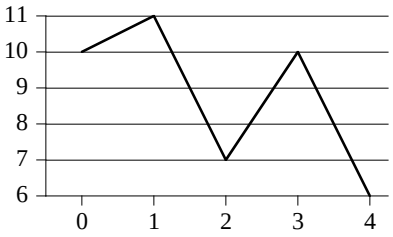
$A[i] =$ (price after day i) $-$ (price after day $(i-1)$) .

[Assuming that we start with a price after day 0, i.e., just before day 1.] Then the nonempty, contiguous subarray with the greatest sum brackets the days that you should have held the stock.

If the maximum subarray is $A[i \,..\, j]$, then should have bought just before day i (i.e., just after day $(i-1)$) and sold just after day j.

Why do we need to find the maximum subarray? Why not just "buy low, sell high"?

- Lowest price might occur *after* the highest price.
- But wouldn't the optimal strategy involve buying at the lowest price *or* selling at the highest price?
- Not necessarily:



Maximum profit is \$3 per share, from buying after day 2 and selling after day 3. Yet lowest price occurs after day 4 and highest occurs after day 1.

Can solve by brute force: check all $\binom{n}{2} = \Theta(n^2)$ subarrays. Can organize the computation so that each subarray $A[i \,..\, j]$ takes $O(1)$ time, given that you've computed $A[i \,..\, j-1]$, so that the brute-force solution takes $\Theta(n^2)$ time.

Solving by divide-and-conquer

Use divide-and-conquer to solve in $O(n \lg n)$ time.

[Maximum subarray might not be unique, though its value is, so we speak of a *maximum subarray, rather than* the *maximum subarray.]*

Subproblem: Find a maximum subarray of $A[low \,..\, high]$.
In original call, $low = 1$, $high = n$.

Divide the subarray into two subarrays of as equal size as possible. Find the midpoint *mid* of the subarrays, and consider the subarrays $A[low\,..\,mid]$ and $A[mid+1\,..\,high]$.

Conquer by finding a maximum subarrays of $A[low\,..\,mid]$ and $A[mid+1\,..\,high]$.

Combine by finding a maximum subarray that crosses the midpoint, and using the best solution out of the three (the subarray crossing the midpoint and the two solutions found in the conquer step).

This strategy works because any subarray must either lie entirely on one side of the midpoint or cross the midpoint.

Finding the maximum subarray that crosses the midpoint

Not a smaller instance of the original problem: has the added restriction that the subarray must cross the midpoint.

Again, could use brute force. If size of $A[low\,..\,high]$ is n, would have $n/2$ choices for left endpoint and $n/2$ choices right endpoint, so would have $\Theta(n^2)$ combinations altogether.

Can solve in linear time.

- Any subarray crossing the midpoint $A[mid]$ is made of two subarrays $A[i\,..\,mid]$ and $A[mid+1\,..\,j]$, where $low \le i \le mid$ and $mid < j \le high$.
- Find maximum subarrays of the form $A[i\,..\,mid]$ and $A[mid+1\,..\,j]$ and then combine them.

Procedure to take array A and indices *low*, *mid*, *high* and return a tuple giving indices of maximum subarray that crosses the midpoint, along with the sum in this maximum subarray:

```
FIND-MAX-CROSSING-SUBARRAY(A, low, mid, high)
  // Find a maximum subarray of the form A[i..mid].
  left-sum = −∞
  sum = 0
  for i = mid downto low
      sum = sum + A[i]
      if sum > left-sum
          left-sum = sum
          max-left = i
  // Find a maximum subarray of the form A[mid+1..j].
  right-sum = −∞
  sum = 0
  for j = mid + 1 to high
      sum = sum + A[j]
      if sum > right-sum
          right-sum = sum
          max-right = j
  // Return the indices and the sum of the two subarrays.
  return (max-left, max-right, left-sum + right-sum)
```

Time: The two loops together consider each index in the range $low, \ldots, high$ exactly once, and each iteration takes $\Theta(1)$ time $\Rightarrow$ procedure takes $\Theta(n)$ time.

Divide-and-conquer procedure for the maximum-subarray problem

FIND-MAXIMUM-SUBARRAY$(A, low, high)$

```
if high == low
    return (low, high, A[low])          // base case: only one element
else mid = ⌊(low + high)/2⌋
    (left-low, left-high, left-sum) =
        FIND-MAXIMUM-SUBARRAY(A, low, mid)
    (right-low, right-high, right-sum) =
        FIND-MAXIMUM-SUBARRAY(A, mid + 1, high)
    (cross-low, cross-high, cross-sum) =
        FIND-MAX-CROSSING-SUBARRAY(A, low, mid, high)
    if left-sum ≥ right-sum and left-sum ≥ cross-sum
        return (left-low, left-high, left-sum)
    elseif right-sum ≥ left-sum and right-sum ≥ cross-sum
        return (right-low, right-high, right-sum)
    else return (cross-low, cross-high, cross-sum)
```

Initial call: FIND-MAXIMUM-SUBARRAY$(A, 1, n)$

- Divide by computing *mid*.
- Conquer by the two recursive calls to FIND-MAXIMUM-SUBARRAY.
- Combine by calling FIND-MAX-CROSSING-SUBARRAY and then determining which of the three results gives the maximum sum.
- Base case is when the subarray has only 1 element.

Analysis

Simplifying assumption: Original problem size is a power of 2, so that all subproblem sizes are integer. *[We made the same simplifying assumption when we analyzed merge sort.]*

Let $T(n)$ denote the running time of FIND-MAXIMUM-SUBARRAY on a subarray of n elements.

Base case: Occurs when *high* equals *low*, so that $n = 1$. The procedure just returns $\Rightarrow T(n) = \Theta(1)$.

Recursive case: Occurs when $n > 1$.

- Dividing takes $\Theta(1)$ time.
- Conquering solves two subproblems, each on a subarray of $n/2$ elements. Takes $T(n/2)$ time for each subproblem $\Rightarrow 2T(n/2)$ time for conquering.
- Combining consists of calling FIND-MAX-CROSSING-SUBARRAY, which takes $\Theta(n)$ time, and a constant number of constant-time tests $\Rightarrow \Theta(n) + \Theta(1)$ time for combining.

Recurrence for recursive case becomes

$$\begin{aligned} T(n) &= \Theta(1) + 2T(n/2) + \Theta(n) + \Theta(1) \\ &= 2T(n/2) + \Theta(n) \qquad \text{(absorb } \Theta(1) \text{ terms into } \Theta(n)) \text{ .} \end{aligned}$$

The recurrence for all cases:

$$T(n) = \begin{cases} \Theta(1) & \text{if } n = 1 \text{ ,} \\ 2T(n/2) + \Theta(n) & \text{if } n > 1 \text{ .} \end{cases}$$

Same recurrence as for merge sort. Can use the master method to show that it has solution $T(n) = \Theta(n \lg n)$.

Thus, with divide-and-conquer, we have developed a $\Theta(n \lg n)$-time solution. Better than the $\Theta(n^2)$-time brute-force solution.

[Can actually solve this problem in $\Theta(n)$ time. See Exercise 4.1-5.]

Strassen's algorithm for matrix multiplication

Input: Two $n \times n$ (square) matrices, $A = (a_{ij})$ and $B = (b_{ij})$.
Output: $n \times n$ matrix $C = (c_{ij})$, where $C = A \cdot B$, i.e.,

$$c_{ij} = \sum_{k=1}^{n} a_{ik} b_{kj}$$

for $i, j = 1, 2, \ldots, n$.

Need to compute n^2 entries of C. Each entry is the sum of n values.

Obvious method

[Using a shorter procedure name than in the book.]

```
SQUARE-MAT-MULT(A, B, n)
  let C be a new n × n matrix
  for i = 1 to n
      for j = 1 to n
          c_ij = 0
          for k = 1 to n
              c_ij = c_ij + a_ik · b_kj
  return C
```

Analysis: Three nested loops, each iterates n times, and innermost loop body takes constant time $\Rightarrow \Theta(n^3)$.

Is $\Theta(n^3)$ the best we can do? Can we multiply matrices in $o(n^3)$ time?

Seems like any algorithm to multiply matrices must take $\Omega(n^3)$ time:

- Must compute n^2 entries.
- Each entry is the sum of n terms.

But with Strassen's method, we can multiply matrices in $o(n^3)$ time.

- Strassen's algorithm runs in $\Theta(n^{\lg 7})$ time.
- $2.80 \leq \lg 7 \leq 2.81$.
- Hence, runs in $O(n^{2.81})$ time.

Simple divide-and-conquer method

As with the other divide-and-conquer algorithms, assume that n is a power of 2.

Partition each of A, B, C into four $n/2 \times n/2$ matrices:

$$A = \begin{pmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{pmatrix}, \quad B = \begin{pmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{pmatrix}, \quad C = \begin{pmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{pmatrix}.$$

Rewrite $C = A \cdot B$ as

$$\begin{pmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{pmatrix} = \begin{pmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{pmatrix} \cdot \begin{pmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{pmatrix},$$

giving the four equations

$$\begin{aligned} C_{11} &= A_{11} \cdot B_{11} + A_{12} \cdot B_{21}, \\ C_{12} &= A_{11} \cdot B_{12} + A_{12} \cdot B_{22}, \\ C_{21} &= A_{21} \cdot B_{11} + A_{22} \cdot B_{21}, \\ C_{22} &= A_{21} \cdot B_{12} + A_{22} \cdot B_{22}. \end{aligned}$$

Each of these equations multiplies two $n/2 \times n/2$ matrices and then adds their $n/2 \times n/2$ products.

Use these equations to get a divide-and-conquer algorithm: *[Using a shorter procedure name than in the book.]*

```
REC-MAT-MULT(A, B, n)
  let C be a new n × n matrix
  if n == 1
      c11 = a11 · b11
  else partition A, B, and C into n/2 × n/2 submatrices
      C11 = REC-MAT-MULT(A11, B11, n/2) + REC-MAT-MULT(A12, B21, n/2)
      C12 = REC-MAT-MULT(A11, B12, n/2) + REC-MAT-MULT(A12, B22, n/2)
      C21 = REC-MAT-MULT(A21, B11, n/2) + REC-MAT-MULT(A22, B21, n/2)
      C22 = REC-MAT-MULT(A21, B12, n/2) + REC-MAT-MULT(A22, B22, n/2)
  return C
```

[The book briefly discusses the question of how to avoid copying entries when partitioning matrices. Can partition matrices without copying entries by instead using index calculations. Identify a submatrix by ranges of row and column matrices

from the original matrix. End up representing a submatrix differently from how we represent the original matrix. The advantage of avoiding copying is that partitioning would take only constant time, instead of $\Theta(n^2)$ time. The result of the asymptotic analysis won't change, but using index calculations to avoid copying gives better constant factors.]

Analysis

Let $T(n)$ be the time to multiply two $n/2 \times n/2$ matrices.

Base case: $n = 1$. Perform one scalar multiplication: $\Theta(1)$.

Recursive case: $n > 1$.

- Dividing takes $\Theta(1)$ time, using index calculations. *[Otherwise, $\Theta(n^2)$ time.]*
- Conquering makes 8 recursive calls, each multiplying $n/2 \times n/2$ matrices $\Rightarrow$ $8T(n/2)$.
- Combining takes $\Theta(n^2)$ time to add $n/2 \times n/2$ matrices four times. *[Doesn't even matter asymptotically whether we use index calculations or copy: would be $\Theta(n^2)$ either way.]*

Recurrence is

$$T(n) = \begin{cases} \Theta(1) & \text{if } n = 1 \ , \\ 8T(n/2) + \Theta(n^2) & \text{if } n > 1 \ . \end{cases}$$

Can use master method to show that it has solution $T(n) = \Theta(n^3)$.
Asymptotically, no better than the obvious method.

Constant factors and recurrences: When setting up recurrences, can absorb constant factors into asymptotic notation, but cannot absorb a constant number of subprobems. Although we absorb the 4 additions of $n/2 \times n/2$ matrices into the $\Theta(n^2)$ time, we cannot lose the 8 in front of the $T(n/2)$ term. If we absorb the constant number of subproblems, then the recursion tree would not be "bushy" and would instead just be a linear chain.

Strassen's method

Idea: Make the recursion tree less bushy. Perform only 7 recursive multiplications of $n/2 \times n/2$ matrices, rather than 8. Will cost several additions of $n/2 \times n/2$ matrices, but just a constant number more $\Rightarrow$ can still absorb the constant factor for matrix additions into the $\Theta(n/2)$ term.

The algorithm:

1. As in the recursive method, partition each of the matrices into four $n/2 \times n/2$ submatrices. Time: $\Theta(1)$.
2. Create 10 matrices $S_1, S_2, \ldots, S_{10}$. Each is $n/2 \times n/2$ and is the sum or difference of two matrices created in previous step. Time: $\Theta(n^2)$ to create all 10 matrices.
3. Recursively compute 7 matrix products $P_1, P_2, \ldots, P_7$, each $n/2 \times n/2$.
4. Compute $n/2 \times n/2$ submatrices of C by adding and subtracting various combinations of the P_i. Time: $\Theta(n^2)$.

Analysis

Recurrence will be

$$T(n) = \begin{cases} \Theta(1) & \text{if } n = 1 \text{ ,} \\ 7T(n/2) + \Theta(n^2) & \text{if } n > 1 \text{ .} \end{cases}$$

By the master method, solution is $T(n) = \Theta(n^{\lg 7})$.

Details

Step 2: Create the 10 matrices

$$\begin{aligned} S_1 &= B_{12} - B_{22} \text{ ,} \\ S_2 &= A_{11} + A_{12} \text{ ,} \\ S_3 &= A_{21} + A_{22} \text{ ,} \\ S_4 &= B_{21} - B_{11} \text{ ,} \\ S_5 &= A_{11} + A_{22} \text{ ,} \\ S_6 &= B_{11} + B_{22} \text{ ,} \\ S_7 &= A_{12} - A_{22} \text{ ,} \\ S_8 &= B_{21} + B_{22} \text{ ,} \\ S_9 &= A_{11} - A_{21} \text{ ,} \\ S_{10} &= B_{11} + B_{12} \text{ .} \end{aligned}$$

Add or subtract $n/2 \times n/2$ matrices 10 times $\Rightarrow$ time is $\Theta(n/2)$.

Step 3: Create the 7 matrices

$$\begin{aligned} P_1 &= A_{11} \cdot S_1 &&= A_{11} \cdot B_{12} - A_{11} \cdot B_{22} \text{ ,} \\ P_2 &= S_2 \cdot B_{22} &&= A_{11} \cdot B_{22} + A_{12} \cdot B_{22} \text{ ,} \\ P_3 &= S_3 \cdot B_{11} &&= A_{21} \cdot B_{11} + A_{22} \cdot B_{11} \text{ ,} \\ P_4 &= A_{22} \cdot S_4 &&= A_{22} \cdot B_{21} - A_{22} \cdot B_{11} \text{ ,} \\ P_5 &= S_5 \cdot S_6 &&= A_{11} \cdot B_{11} + A_{11} \cdot B_{22} + A_{22} \cdot B_{11} + A_{22} \cdot B_{22} \text{ ,} \\ P_6 &= S_7 \cdot S_8 &&= A_{12} \cdot B_{21} + A_{12} \cdot B_{22} - A_{22} \cdot B_{21} - A_{22} \cdot B_{22} \text{ ,} \\ P_7 &= S_9 \cdot S_{10} &&= A_{11} \cdot B_{11} + A_{11} \cdot B_{12} - A_{21} \cdot B_{11} - A_{21} \cdot B_{12} \text{ .} \end{aligned}$$

The only multiplications needed are in the middle column; right-hand column just shows the products in terms of the original submatrices of A and B.

Step 4: Add and subtract the P_i to construct submatrices of C:

$$\begin{aligned} C_{11} &= P_5 + P_4 - P_2 + P_6 \text{ ,} \\ C_{12} &= P_1 + P_2 \text{ ,} \\ C_{21} &= P_3 + P_4 \text{ ,} \\ C_{22} &= P_5 + P_1 - P_3 - P_7 \text{ .} \end{aligned}$$

To see how these computations work, expand each right-hand side, replacing each P_i with the submatrices of A and B that form it, and cancel terms: *[We expand out all four right-hand sides here. You might want to do just one or two of them, to convince students that it works.]*

$$
\begin{array}{llll}
A_{11}\cdot B_{11} + A_{11}\cdot B_{22} + A_{22}\cdot B_{11} + A_{22}\cdot B_{22} & & & \\
\quad - A_{22}\cdot B_{11} & + A_{22}\cdot B_{21} & & \\
- A_{11}\cdot B_{22} & & - A_{12}\cdot B_{22} & \\
& - A_{22}\cdot B_{22} - A_{22}\cdot B_{21} + A_{12}\cdot B_{22} + A_{12}\cdot B_{21} & & \\
\hline
A_{11}\cdot B_{11} & & & + A_{12}\cdot B_{21}
\end{array}
$$

$$
\begin{array}{ll}
A_{11}\cdot B_{12} - A_{11}\cdot B_{22} & \\
\quad + A_{11}\cdot B_{22} + A_{12}\cdot B_{22} & \\
\hline
A_{11}\cdot B_{12} & + A_{12}\cdot B_{22}
\end{array}
$$

$$
\begin{array}{ll}
A_{21}\cdot B_{11} + A_{22}\cdot B_{11} & \\
\quad - A_{22}\cdot B_{11} + A_{22}\cdot B_{21} & \\
\hline
A_{21}\cdot B_{11} & + A_{22}\cdot B_{21}
\end{array}
$$

$$
\begin{array}{llll}
A_{11}\cdot B_{11} + A_{11}\cdot B_{22} + A_{22}\cdot B_{11} + A_{22}\cdot B_{22} & & & \\
\quad - A_{11}\cdot B_{22} & + A_{11}\cdot B_{12} & & \\
& - A_{22}\cdot B_{11} & - A_{21}\cdot B_{11} & \\
- A_{11}\cdot B_{11} & & - A_{11}\cdot B_{12} + A_{21}\cdot B_{11} + A_{21}\cdot B_{12} & \\
\hline
& A_{22}\cdot B_{22} & & + A_{21}\cdot B_{12}
\end{array}
$$

Theoretical and practical notes

Strassen's algorithm was the first to beat $\Theta(n^3)$ time, but it's not the asymptotically fastest known. A method by Coppersmith and Winograd runs in $O(n^{2.376})$ time.

Practical issues against Strassen's algorithm:

- Higher constant factor than the obvious $\Theta(n^3)$-time method.
- Not good for sparse matrices.
- Not numerically stable: larger errors accumulate than in the obvious method.
- Submatrices consume space, especially if copying.

Numerical stability problem is not as bad as previously thought. And can use index calculations to reduce space requirement.

Various researchers have tried to find the crossover point, where Strassen's algorithm runs faster than the obvious $\Theta(n^3)$-time method. Analyses (that ignore caches and hardware pipelines) have produced crossover points as low as $n = 8$, and experiments have found crossover points as low as $n = 400$.

Substitution method

1. Guess the solution.
2. Use induction to find the constants and show that the solution works.

Example

$$T(n) = \begin{cases} 1 & \text{if } n = 1 \,, \\ 2T(n/2) + n & \text{if } n > 1 \,. \end{cases}$$

1. *Guess:* $T(n) = n \lg n + n$. *[Here, we have a recurrence with an exact function, rather than asymptotic notation, and the solution is also exact rather than asymptotic. We'll have to check boundary conditions and the base case.]*
2. *Induction:*

   Basis: $n = 1 \Rightarrow n \lg n + n = 1 = T(n)$

   Inductive step: Inductive hypothesis is that $T(k) = k \lg k + k$ for all $k < n$. We'll use this inductive hypothesis for $T(n/2)$.

$$\begin{aligned} T(n) &= 2T\left(\frac{n}{2}\right) + n \\ &= 2\left(\frac{n}{2}\lg\frac{n}{2} + \frac{n}{2}\right) + n \qquad \text{(by inductive hypothesis)} \\ &= n \lg \frac{n}{2} + n + n \\ &= n(\lg n - \lg 2) + n + n \\ &= n \lg n - n + n + n \\ &= n \lg n + n \,. \end{aligned}$$

■

Generally, we use asymptotic notation:

- We would write $T(n) = 2T(n/2) + \Theta(n)$.
- We assume $T(n) = O(1)$ for sufficiently small n.
- We express the solution by asymptotic notation: $T(n) = \Theta(n \lg n)$.
- We don't worry about boundary cases, nor do we show base cases in the substitution proof.
  - $T(n)$ is always constant for any constant n.
  - Since we are ultimately interested in an asymptotic solution to a recurrence, it will always be possible to choose base cases that work.
  - When we want an asymptotic solution to a recurrence, we don't worry about the base cases in our proofs.
  - When we want an exact solution, then we have to deal with base cases.

For the substitution method:

- Name the constant in the additive term.
- Show the upper (O) and lower (Ω) bounds separately. Might need to use different constants for each.

Example

$T(n) = 2T(n/2) + \Theta(n)$. If we want to show an upper bound of $T(n) = 2T(n/2) + O(n)$, we write $T(n) \le 2T(n/2) + cn$ for some positive constant c.

1. ***Upper bound:***

   Guess: $T(n) \le dn \lg n$ for some positive constant d. We are given c in the recurrence, and we get to choose d as any positive constant. It's OK for d to depend on c.

   Substitution:

$$\begin{aligned} T(n) &\le 2T(n/2) + cn \\ &= 2\left(d\frac{n}{2}\lg\frac{n}{2}\right) + cn \\ &= dn\lg\frac{n}{2} + cn \\ &= dn\lg n - dn + cn \\ &\le dn\lg n \qquad \text{if } -dn + cn \le 0, \\ &\qquad\qquad\qquad\qquad d \ge c \end{aligned}$$

   Therefore, $T(n) = O(n \lg n)$.

2. ***Lower bound:*** Write $T(n) \ge 2T(n/2) + cn$ for some positive constant c.

   Guess: $T(n) \ge dn \lg n$ for some positive constant d.

   Substitution:

$$\begin{aligned} T(n) &\ge 2T(n/2) + cn \\ &= 2\left(d\frac{n}{2}\lg\frac{n}{2}\right) + cn \\ &= dn\lg\frac{n}{2} + cn \\ &= dn\lg n - dn + cn \\ &\ge dn\lg n \qquad \text{if } -dn + cn \ge 0, \\ &\qquad\qquad\qquad\qquad d \le c \end{aligned}$$

   Therefore, $T(n) = \Omega(n \lg n)$.

Therefore, $T(n) = \Theta(n \lg n)$. *[For this particular recurrence, we can use $d = c$ for both the upper-bound and lower-bound proofs. That won't always be the case.]* ■

Make sure you show the same *exact* form when doing a substitution proof.

Consider the recurrence

$$T(n) = 8T(n/2) + \Theta(n^2) .$$

For an upper bound:

$$T(n) \le 8T(n/2) + cn^2 .$$

Guess: $T(n) \le dn^3$.

$$\begin{aligned} T(n) &\le 8d(n/2)^3 + cn^2 \\ &= 8d(n^3/8) + cn^2 \\ &= dn^3 + cn^2 \\ &\not\le dn^3 \qquad \text{doesn't work!} \end{aligned}$$

Remedy: *Subtract off* a lower-order term.

Guess: $T(n) \le dn^3 - d'n^2$.

$$\begin{aligned} T(n) &\le 8(d(n/2)^3 - d'(n/2)^2) + cn^2 \\ &= 8d(n^3/8) - 8d'(n^2/4) + cn^2 \\ &= dn^3 - 2d'n^2 + cn^2 \\ &= dn^3 - d'n^2 - d'n^2 + cn^2 \\ &\le dn^3 - d'n^2 \qquad \text{if } -d'n^2 + cn^2 \le 0\,, \\ &\qquad\qquad\qquad\qquad\qquad\quad d' \ge c \end{aligned}$$

Be careful when using asymptotic notation.

The false proof for the recurrence $T(n) = 4T(n/4) + n$, that $T(n) = O(n)$:

$$\begin{aligned} T(n) &\le 4(c(n/4)) + n \\ &\le cn + n \\ &= O(n) \qquad \text{wrong!} \end{aligned}$$

Because we haven't proven the *exact form* of our inductive hypothesis (which is that $T(n) \le cn$), this proof is false.

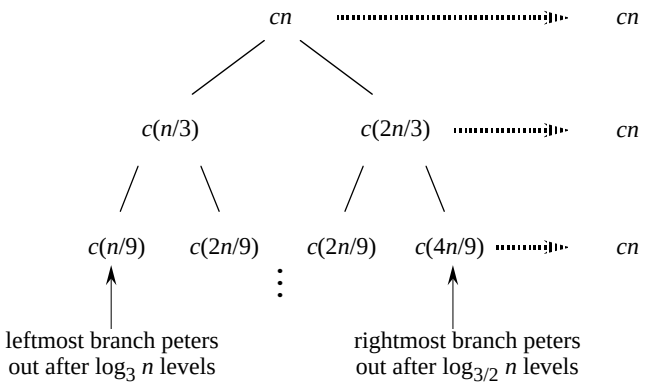
Recursion trees

Use to generate a guess. Then verify by substitution method.

Example

$T(n) = T(n/3) + T(2n/3) + \Theta(n)$.
For upper bound, rewrite as $T(n) \le T(n/3) + T(2n/3) + cn$; for lower bound, as $T(n) \ge T(n/3) + T(2n/3) + cn$.

By summing across each level, the recursion tree shows the cost at each level of recursion (minus the costs of recursive calls, which appear in subtrees):



- There are $\log_3 n$ full levels, and after $\log_{3/2} n$ levels, the problem size is down to 1.
- Each level contributes $\le cn$.
- Lower bound guess: $\ge dn \log_3 n = \Omega(n \lg n)$ for some positive constant d.

- Upper bound guess: $\le dn \log_{3/2} n = O(n \lg n)$ for some positive constant d.
- Then *prove* by substitution.

1. ***Upper bound:***

   Guess: $T(n) \le dn \lg n$.

   Substitution:

$$\begin{aligned} T(n) &\le T(n/3) + T(2n/3) + cn \\ &\le d(n/3)\lg(n/3) + d(2n/3)\lg(2n/3) + cn \\ &= (d(n/3)\lg n - d(n/3)\lg 3) \\ &\qquad + (d(2n/3)\lg n - d(2n/3)\lg(3/2)) + cn \\ &= dn\lg n - d((n/3)\lg 3 + (2n/3)\lg(3/2)) + cn \\ &= dn\lg n - d((n/3)\lg 3 + (2n/3)\lg 3 - (2n/3)\lg 2) + cn \\ &= dn\lg n - dn(\lg 3 - 2/3) + cn \\ &\le dn\lg n \qquad \text{if } -dn(\lg 3 - 2/3) + cn \le 0 , \\ &\qquad\qquad\qquad\qquad\qquad\qquad d \ge \frac{c}{\lg 3 - 2/3} . \end{aligned}$$

   Therefore, $T(n) = O(n \lg n)$.

   Note: Make sure that the symbolic constants used in the recurrence (e.g., c) and the guess (e.g., d) are different.

2. ***Lower bound:***

   Guess: $T(n) \ge dn \lg n$.

   Substitution: Same as for the upper bound, but replacing $\le$ by $\ge$. End up needing

$$0 < d \le \frac{c}{\lg 3 - 2/3} .$$

   Therefore, $T(n) = \Omega(n \lg n)$.

Since $T(n) = O(n \lg n)$ and $T(n) = \Omega(n \lg n)$, we conclude that $T(n) = \Theta(n \lg n)$. ■

Master method

Used for many divide-and-conquer recurrences of the form

$$T(n) = aT(n/b) + f(n) ,$$

where $a \ge 1$, $b > 1$, and $f(n) > 0$.

Based on the ***master theorem*** (Theorem 4.1).

Compare $n^{\log_b a}$ vs. $f(n)$:

Case 1: $f(n) = O(n^{\log_b a - \epsilon})$ for some constant $\epsilon > 0$.
($f(n)$ is polynomially smaller than $n^{\log_b a}$.)
Solution: $T(n) = \Theta(n^{\log_b a})$.
(Intuitively: cost is dominated by leaves.)

Case 2: $f(n) = \Theta(n^{\log_b a} \lg^k n)$, where $k \geq 0$.
[This formulation of Case 2 is more general than in Theorem 4.1, and it is given in Exercise 4.6-2.]
($f(n)$ is within a polylog factor of $n^{\log_b a}$, but not smaller.)
Solution: $T(n) = \Theta(n^{\log_b a} \lg^{k+1} n)$.
(Intuitively: cost is $n^{\log_b a} \lg^k n$ at each level, and there are $\Theta(\lg n)$ levels.)
Simple case: $k = 0 \Rightarrow f(n) = \Theta(n^{\log_b a}) \Rightarrow T(n) = \Theta(n^{\log_b a} \lg n)$.

Case 3: $f(n) = \Omega(n^{\log_b a + \epsilon})$ for some constant $\epsilon > 0$ and $f(n)$ satisfies the regularity condition $af(n/b) \leq cf(n)$ for some constant $c < 1$ and all sufficiently large n.
($f(n)$ is polynomially greater than $n^{\log_b a}$.)
Solution: $T(n) = \Theta(f(n))$.
(Intuitively: cost is dominated by root.)

What's with the Case 3 regularity condition?

- Generally not a problem.
- It always holds whenever $f(n) = n^k$ and $f(n) = \Omega(n^{\log_b a + \epsilon})$ for constant $\epsilon > 0$. *[Proving this makes a nice homework exercise. See below.]* So you don't need to check it when $f(n)$ is a polynomial.

[Here's a proof that the regularity condition holds when $f(n) = n^k$ and $f(n) = \Omega(n^{\log_b a + \epsilon})$ for constant $\epsilon > 0$.

Since $f(n) = \Omega(n^{\log_b a + \epsilon})$ and $f(n) = n^k$, we have that $k > \log_b a$. Using a base of b and treating both sides as exponents, we have $b^k > b^{\log_b a} = a$, and so $a/b^k < 1$. Since a, b, and k are constants, if we let $c = a/b^k$, then c is a constant strictly less than 1. We have that $af(n/b) = a(n/b)^k = (a/b^k)n^k = cf(n)$, and so the regularity condition is satisfied.]

Examples

- $T(n) = 5T(n/2) + \Theta(n^2)$
  $n^{\log_2 5}$ vs. n^2
  Since $\log_2 5 - \epsilon = 2$ for some constant $\epsilon > 0$, use Case 1 $\Rightarrow T(n) = \Theta(n^{\lg 5})$

- $T(n) = 27T(n/3) + \Theta(n^3 \lg n)$
  $n^{\log_3 27} = n^3$ vs. $n^3 \lg n$
  Use Case 2 with $k = 1 \Rightarrow T(n) = \Theta(n^3 \lg^2 n)$

- $T(n) = 5T(n/2) + \Theta(n^3)$
  $n^{\log_2 5}$ vs. n^3
  Now $\lg 5 + \epsilon = 3$ for some constant $\epsilon > 0$
  Check regularity condition (don't really need to since $f(n)$ is a polynomial):
  $af(n/b) = 5(n/2)^3 = 5n^3/8 \leq cn^3$ for $c = 5/8 < 1$
  Use Case 3 $\Rightarrow T(n) = \Theta(n^3)$

- $T(n) = 27T(n/3) + \Theta(n^3/\lg n)$
  $n^{\log_3 27} = n^3$ vs. $n^3/\lg n = n^3 \lg^{-1} n \neq \Theta(n^3 \lg^k n)$ for any $k \geq 0$.
  Cannot use the master method.